PyRetinotopy

Retinotopic visual stimulation with PsychoPy

Project and Experiment Guide / English Edition

Author and responsible investigator	Version and revision date
Hiroshi Ban	v0.4.0 / 2026-11-10

Purpose

PyRetinotopy provides flexible retinotopic visual stimuli for studying cortical representations of visual-field position with fMRI. It supports monocular and binocular display arrangements, with particular attention to presentation timing. The stimuli can also be used in MEG, EEG, and visual psychophysics.

Features carried forward from the original package

PyRetinotopy adapts the MATLAB Psychtoolbox 3 Retinotopy stimulus set (https://github.com/hiroshiban/Retinotopy), developed and publicly released by Hiroshi Ban, to Python and PsychoPy. One program combines a broad stimulus repertoire, object-image and random-dot carriers, participant-specific settings, stereoscopic layouts, and timing relative to a single run onset.

Stimuli	Operation
14 basic modes × 5 carrier/task options = 70 modes	v0.4.0 / 2026-11-10
Single display, side-by-side, top/bottom, anaglyph, interleaved, dual display, and quad buffer	Native NumPy image banks and complete object-movie preparation before acquisition

Using this guide

The guide introduces the project, then covers setup, acquisition, timing checks, citation, and acknowledgments. Commands use Windows PowerShell. Replace example calibration values and stimulus parameters with those measured and selected for your experiment.

PyRetinotopy was developed with support from OpenAI generative AI tools, including Codex.

Hiroshi Ban, author and responsible investigator of PyRetinotopy, reviews the transfer of functionality from the original MATLAB/PTB3 Retinotopy and performs operational checks throughout development. He is responsible for the software and its release.

02 Quick start

Use this procedure to prepare retinotopic stimuli on a local experimental computer, start presentation from a scanner signal, and check timing after acquisition.

Work through these steps

1 Check the environment	2 Calibrate the setup	3 Prepare before acquisition	4 Run and inspect
Run list and the short demo with the PsychoPy interpreter	Set screen dimensions, viewing distance, refresh, and eye assignment	Provide the image bank and prepare the object-movie cache	Wait for the start signal; inspect summary and boundary errors afterward

Common PowerShell setup

The PowerShell example assumes a checkout at E:\PyRetinotopy\PyRetinotopy. Adjust the project path and PsychoPy interpreter path for your installation. The pyrt function is a shortcut for this terminal session.

```
Set-Location 'E:\PyRetinotopy\PyRetinotopy'
$retinoPython = 'C:\Program Files\PsychoPy\python.exe'
function pyrt {
  & $retinoPython -m pyretinotopy @args
}
pyrt list
pyrt validate --config examples\ccwf.json
pyrt run --config examples\smoke.json
```

smoke.json is a short, automatically starting 640×480 demo. Repeating an acquisition archives its complete previous directory with a numbered suffix. Use --run N for a distinct scan.

Guide contents

3: Modes / 4: Starting positions / 5: Settings / 6: Displays / 7: Images / 8: Acquisition / 9: Runs and retries / 10: PTB3 controls / 11: Timing / 12: Results / 13: Troubleshooting / 14: Citation / 15: Acknowledgments

03 Choose a stimulus mode

Fourteen basic modes combine with five carrier/task suffixes to produce 70 mode names. Stimulus geometry and display layout are configured independently.

Basic mode	Description
ccw / cw	Counter-clockwise / clockwise rotating wedge
exp / cont	Expanding / contracting annulus
bar	Sweeping bars in multiple directions for pRF mapping
ccwexp / ccwcont	Counter-clockwise wedge + expanding / contracting annulus
cwexp / cwcont	Clockwise wedge + expanding / contracting annulus
multifocal	Polar-angle × eccentricity regions driven by an m-sequence or a supplied design
meridian	Opposing wedges along the horizontal / vertical meridian
lgn	Alternating visual hemifields for LGN localization
hrf	Stimulus/rest cycles for HRF estimation
localizer	A selected eccentricity band and its complement

Suffix	Carrier / task	Example
none	Color checkerboard / stimulus luminance-decrement detection	ccw
f	Color checkerboard / fixation task	ccwf
d	Random-dot stereogram / depth-change detection	ccwd
df	Random-dot stereogram / fixation task	ccwdf
i	Object images / fixation task	ccwi

Timing parameters

For wedge, annulus, and HRF protocols, active protocol duration is cycle_s × repeats. Bars use cycle_s × number of directions × repeats; multifocal uses trial_s × sequence length; meridian, LGN, and localizer use block_s × 2 × repeats. Add initial_fixation_s and final_fixation_s to obtain total run duration.

Combined wedge/annulus protocols use independent polar_* and eccentricity_* cycles. Their total durations must match. rest_s and related fields are included within each cycle, block, or trial. Check duration_s in the validate output.

For standalone JSON settings, create a configuration per protocol with init --mode. Overriding --mode on an existing JSON changes its name, without resetting the other fields. For inherited settings, specify --subject and --mode.

04 Starting positions and PTB3 compatibility

Version 0.3.0 follows the original image-coordinate angles and reversed image lists. Starting and scan positions were checked for all 70 modes. The default initial fixation lasts four seconds.

Stimulus	Default first position and rule
cw / ccw	cw is centered upward at 90 degrees; ccw at 102 degrees, 12 degrees counter-clockwise from up. PTB3 presents all 30 ccw images in reverse order.
exp / cont	exp starts centrally; cont starts peripherally after ten seconds of rest. Standalone annuli have 25 positions.
Dual / bar	Dual annuli have 16 positions, a two-second dwell and one-degree nominal width. The first bar sweeps from right to left.
Multifocal / LGN	Multifocal regions follow clockwise image angles. LGN starts on the left, then alternates with its horizontal reflection.
Meridian / HRF / localizer	Start with horizontal meridians, the whole field, and the target eccentricity band, respectively.

Angles and compatibility settings

stimulus.spatial_convention defaults to "ptb3". Configuration start_angle_deg and dual_ring_start_angle_deg use the original image convention: right is zero, clockwise is positive. Internal components and exported windows are converted to visual coordinates with positive y upward.

multifocal_indexing defaults to "corrected", giving 36 unique regions. Select "ptb3" to reproduce the original zero-phase overlapping region IDs (phase zero and width 360 only). Record this choice with the saved configuration.

To replay Python versions through 0.2.2, use spatial_convention "legacy-python". An old acquisition config.json without this field is recognized automatically when its sibling summary.json records version 0.2.2 or earlier. Ordinary settings files adopt the corrected defaults.

All positions were compared with a separate implementation of the source formulas in continuous coordinates. This is not a MATLAB framebuffer comparison: pixel rounding, task sequences and random carriers may differ. docs/PTB3_PARITY.ja.md records formulas, comparisons and limits.

05 Environment and participant settings

Tested environment and installation

The tested environment is Windows, PsychoPy 2024.2.4, and Python 3.10.11. Core dependencies are NumPy and Pillow; presentation requires PsychoPy. MATLAB is not required. In an existing working PsychoPy environment, pip install -e . installs the source checkout.

For full-screen acquisition on Windows, install the GLFW backend from the project directory using the command below.

```
& $retinoPython -m pip install --target optional-libs --no-deps `
  psychopy-glfw==0.0.2 glfw==2.8.0
pyrt init-subject HB --mode ccwf
```

subjects/_DEFAULT_/ corresponds to the original settings directory. The loader reads common.json, retinotopy_display.json, the family-specific c_*.json, and an optional mode-name JSON. It then applies the same files from subjects/HB/. Participant files need contain only the fields to change.

Edit JSON to match the apparatus and experiment

Section	Parameters to set
subject / run / seed	Participant ID, acquisition number, and random seed. Save enough information to identify each run.
display	mode, size_px, screens, width_cm, distance_cm, refresh_hz, backend, full_screen.
timing	Initial/final fixation, cycles, repeats, rest, start method, trigger and response keys.
stimulus	Radii, wedge width, bar directions, colors, disparity, dots, and object images.
task	Fixation type/size/color, fixation_frequency, duration_s, schedule and response window.

Visual-angle and screen calibration

width_cm is the physical width corresponding to all of size_px; distance_cm is the eye-to-display distance. For side-by-side presentation, enter the full screen width. Pixels per degree are size_px[0] / width_cm × distance_cm × tan(1 degree). A stimulus that does not fit its eye viewport is rejected.

The runner rejects an actual framebuffer size different from size_px, or an unstable refresh measurement. Check OS scaling, resolution, and display settings. Physical dimensions in example files are not measured calibration values.

```
pyrt validate --subject HB --mode ccwf
pyrt preview --subject HB --mode ccwf --output preview.png
```

Use --subject for inheritance. --config FILE is a standalone JSON and does not search other default files; use it to replay saved acquisition settings. See subjects/README.md.

validate checks configuration and scheduling. Calibration, optical timing, eye separation, and performance still require apparatus-specific checks.

06 Match the monocular or binocular display

display.mode	Layout and checks
mono	One eye image in one window.
parallel / cross	Side-by-side; parallel places the left-eye image on the left, cross reverses it.
dualparallel / dualcross	One wide framebuffer split left/right. Check the OS display arrangement.
dual	Two windows on distinct screen IDs. Both swap timestamps are recorded.
topbottom / bottomtop	Vertical split; topbottom places the left-eye image at the top, bottomtop reverses it.
redgreen / greenred	Luminance projected into red/green channels; greenred reverses eye assignment.
redblue / bluered	Luminance projected into red and blue/cyan-side channels with original gains.
interleavedline / interleavedcolumn	Left eye on even rows/columns, right eye on odd ones, using zero-based indexing. Check the apparatus origin.
shutter	Quad-buffer stereo; requires compatible GPU, driver, and display hardware.

Offsets and mirroring

eye_offsets_px contains [x, y] per eye, with positive x to the right and positive y upward. mirror_x and mirror_y reflect each complete eye scene, including fixation, markers, and offsets. Check each eye separately. The program does not squeeze an image to fit an eye viewport.

Before using stereoscopic presentation

1. Occlude one eye at a time to verify assignment. 2. Use fixation and the visual-field boundary to adjust position, size, and mirroring. 3. Confirm that negative RDS disparity is perceived as near. 4. Measure luminance and leakage through the glasses.

dual issues sequential swaps and does not guarantee hardware synchronization. Inspect eye_skew_s in frames.tsv and measure both eyes optically when needed. shutter fails explicitly if quad-buffer stereo is unavailable.

Initial settings for apparatus qualification

Start with full_screen=true, backend="glfw", high_priority=true, and realtime_priority=false. The highest priority is not always faster. gamma applies inverse power-law correction per channel; arbitrary hardware gamma LUTs are not automatically migrated.

Dedicated PROPixx / Datapixx control is not implemented. Do not assume equivalence with ordinary quad-buffer or split-screen presentation.

07 Prepare object images before acquisition

An image bank independent of MATLAB

For object stimuli, provide assets/object-images/images.npy and manifest.json. The standard bank contains 7,493 uint8 images with axes image × height × width × RGBA. Conversion verifies every pixel and alpha value and records the shape and pixel digest in manifest.json. Native presentation does not use MAT files or SciPy/h5py.

Place images.npy and manifest.json together. Large image data are excluded from source Git tracking. A directory of ordinary image files such as PNG is also supported as object_source.

Optional one-time conversion from the legacy bank

```
pyrt convert-images `
  ..\Retinotopy\object_images\object_image_database.mat `
  --output assets\object-images
```

This conversion requires SciPy/h5py; add them only when needed with pip install -e .[legacy]. The input is read-only, and existing images.npy files are never overwritten.

Paths are relative to the defining JSON

Relative object_source and object_cache_dir paths are resolved from the JSON file that defines each value. From subjects/HB/ccwi.json, the following stimulus-section excerpt points to the project root.

```
"object_source": "../../assets/object-images/manifest.json",
"object_cache_dir": "../../cache/object-movies",
"object_count": 120,
"object_update_s": 0.25,
"object_texture_slots": 4
```

Finish synthesis before waiting for the trigger

```
pyrt prepare-objects --config examples\object-images-fullscreen.json
pyrt run --config examples\object-images-fullscreen.json
```

All object transformations, rotations, noise, gamma correction, and anaglyph encoding are prepared before acquisition. Unchanged settings and seed reuse the cache; changed conditions produce a separate cache. run also prepares any missing movie before initializing the display.

A 360-second stimulus with 1031×1031 eye images and 250 ms updates needs approximately 9.2 GB of movie cache, plus approximately 1.96 GB for the source bank. During acquisition, only prepared-frame prefetch and GPU transfers occur. If preparation falls behind, acquisition stops and saves its logs.

08 Run an acquisition

1 Check settings and preparation

Verify participant ID, run, seed, display calibration, cycle count and total duration, and trigger/response keys. Finish prepare-objects for image stimuli. Avoid updates, notifications, and competing GPU applications during acquisition. Qualify the same settings at normal run length before using them experimentally.

```
pyrt validate --subject HB --mode ccwf
pyrt run --subject HB --mode ccwf
```

2 Supply the start signal

timing.start	Start action
trigger (default)	A key in trigger_keys; defaults are t or 5.
key	Space or Enter.
mouse	Left mouse click.
immediate	Automatic start after setup; intended for demos/performance tests.

skip_triggers=0 starts on the first pulse; 4 ignores four pulses and starts on the fifth. Fixation is visible while waiting. The first measured flip after the accepted signal defines run time zero; initial fixation is part of the run.

Scanner pulses can be supplied as keyboard events through a USB or similar interface. Direct parallel-port voltage input is not implemented. events.tsv preserves the difference between trigger time and the first flip.

3 Responses and aborting

Default response keys are r / g; Escape or Q aborts. Reaction time is measured from the actual target-onset flip. An abort presents a terminal blank and saves the acquired portion of the logs; unreached boundaries are n/a. Escape or Q also aborts while waiting for a start signal.

4 Preserve the results and prepare the next run

Results are stored by participant and date, for example subjects/HB/results/261106/ccwf_run-01/. YYMMDD is the experimental computer's local date at run initialization. Use --run N for a distinct scan; repeating the same number archives the previous attempt. init-subject can add settings to an ID directory that already contains results.

Inherited --subjects-dir settings and explicit JSON inside an ID directory use that participant root for results. Example JSON files use the project subjects directory. Only an explicit --output-root ROOT selects the separate ROOT/sub-ID/mode_run-NN/ layout. Existing data directories are not moved.

```
pyrt session examples\session.json
```

session.json lists configurations or participant/mode selections and run numbers. Set start=trigger in each configuration to wait separately for every run. An abort stops the remaining session.

09 Run numbers and repeat attempts

Use --run to distinguish repeated scans of the same stimulus. The argument overrides the JSON run value and is recorded in the saved configuration, summary and directory name.

```
pyrt run --subject HB --mode ccw --run 1
pyrt run --subject HB --mode ccw --run 2
# Repeat run 1 after motion or a response problem
pyrt run --subject HB --mode ccw --run 1
```

Repeating the same participant, date, mode and run archives the complete previous directory with a numbered suffix. The latest attempt uses the original directory name. Other run numbers and earlier backups retain their contents and names.

```
subjects/HB/results/261106/
  ccw_run-01/          # latest attempt
  ccw_run-01_old-01/   # first previous attempt
  ccw_run-01_old-02/   # next previous attempt
  ccw_run-02/          # a separate scan
```

Setting / option	Behavior
--if-exists backup	Default: archive previous results, then retry.
--if-exists error	Cancel without changes if results exist.
--no-confirm	Skip operator review; keep the scanner-trigger wait.

operator_confirm=true displays the participant, mode and run before backup or preparation. Cancelling leaves previous data untouched. Completed, aborted and failed attempts are preserved; use summary.json status and logs to select the accepted acquisition.

acquisition.json records a unique ID and the previous backup path. An OS lock is held through final saving to prevent moving an active run. The reusable .lock file may remain after exit; the OS releases the lock when the process terminates.

Coder and sessions use the same procedure. Set a run number for each scan in a session. Duplicate destinations within a single session definition are rejected.

10 PTB3 fixation, task and display controls

PTB3	fixation_type	Shape
1	circle	Circular target.
2	cross	Original framed cross with tick marks, rather than a filled square.
3	concentric	Central disk and outer quadrants separated by gaps.

Set task.fixation_radius_px and fixation_color. The dim color defaults to [64,64,64]/255. A filled square remains available as a Python extension.

```
"task": {
  "fixation_type": "concentric",
  "fixation_radius_px": 12,
  "schedule": "ptb3",
  "fixation_frequency": 4,
  "duration_s": 0.3
}
```

The ptb3 fixation schedule retains fixtaskfreq probability, duration slots and an intervening normal slot. Initial/final fixation and rest are eligible. Stimulus tasks use half-position or shorter color/depth windows. The interval option uses the earlier interval_s and duration_s controls.

display field	Purpose
background_type = grid	Shared binocular peripheral guides, patch sizes/counts/colors and a clear central aperture.
marker_timing = half	Pulse for half a position; fixed uses marker_duration_s.
gamma_lut / gamma_lut_right	Native N-by-3 RGB tables; right selects physical display 2 in dual mode.
use_measured_refresh = true	Select measured refresh, as with PTB3 force_frame_rate=0.

Checker and RDS updates start at first active presentation and pause during rest. RDS cycles opposing checker-plane disparities. docs/PTB3_FEATURES.ja.md and PTB3_PARAMETER_MAP.ja.md record all controls, legacy Python replay and remaining hardware limits.

11 Keep boundaries on cumulative run time

All protocols keep the first actual run-flip time, t0, fixed. A measured block end never becomes the next block origin. With 4 seconds of initial fixation and 60-second cycles, the absolute targets are:

Cycle	Fixed target relative to run onset
1	t0 + 4 s
2	t0 + 64 s
3	t0 + 124 s
4	t0 + 184 s
5	t0 + 244 s
6	t0 + 304 s

The table gives cumulative target times for 4 seconds of initial fixation and 60-second cycles. Actual transitions are saved in block_timing.tsv for each acquisition. Inspect planned_onset_s, actual_onset_s, and error_s to check whether errors increase toward the end of a run.

Choose a state for each flip

The loop maps PsychoPy's predicted next-flip time to a nominal refresh slot on the cumulative run clock. A slightly faster display may hold a state; a slower display advances to the state appropriate to the current time. The nominal frame number is not forced to increase at every physical flip, avoiding drift caused by small refresh-rate differences.

Wedges, annuli, and HRF use cycle boundaries; bars use sweeps; multifocal uses trials; meridian, LGN, and localizer use blocks. Combined wedge/annulus protocols compute both independent cycles from the same t0. Aperture steps and tasks also use cumulative run time.

Read block_timing.tsv

Column	Meaning
planned_onset_s	Continuous target time relative to run onset.
scheduled_onset_s	Target assigned to a nominal refresh slot.
actual_onset_s	Measured flip that displayed the corresponding state.
error_s / frame	Actual minus planned time in seconds / physical flip index.

Refresh quantization and OS/GPU jitter remain. A late frame cannot be undone, but its delay is not added to later deadlines. PsychoPy synchronization timestamps are distinct from light reaching the eye; verify optical timing on the intended apparatus.

12 Inspect results and export for analysis

Saved file	What to inspect
acquisition.json	Unique acquisition ID, run number and previous backup path.
config.json	Effective acquisition settings, seed, and absolute image/cache paths.
summary.json	Completion/abort/failure status, error, flip statistics, boundary error, and environment.
frames.tsv	Planned and actual flips, intervals, states, carrier updates, CPU costs, and inter-eye skew.
block_timing.tsv	Planned / actual / error for cycles, sweeps, trials, and blocks.
events.tsv / events.json	Stimulus, blank, trigger, response, target, and termination times with descriptions.
targets.tsv / responses.tsv	Planned/actual target onset, response classification, and reaction time.
multifocal_design.npy	The region × time design actually used for a multifocal run.

Post-acquisition checks

First inspect status and error, then timing.long_intervals, estimated_missed_frames, and max_interval_s. A long interval exceeds 1.5 nominal refresh periods: 25 ms at 60 Hz. Missed-frame counts are estimates from flip intervals, not optical measurements.

Next plot error_s in block_timing.tsv against time. Check both the maximum and whether error grows later in the run. update_cpu_s, draw_cpu_s, and upload_cpu_s in frames.tsv help locate processing costs. All task targets being missed is expected in unattended performance tests.

Export analysis apertures

```
pyrt export --config examples\barf.json `
  --output bar-windows.npz --tr 2 --resolution 128
$resultDay = Get-Date -Format yyMMdd
$runDir = "subjects\HB\results\$resultDay\ccwf_run-01"
pyrt export --config "$runDir\config.json" `
  --frames "$runDir\frames.tsv" `
  --output actual-windows.npz --prt actual.prt --tr 2
```

The first command exports the planned apertures; the second uses actual flips. Actual-flip export integrates exposure within each TR while holding the previous state until its next flip. NPZ axes are time × y × x, with upper rows representing the upper visual field. Optional MAT-compatible export uses y × x × time.

Aperture arrays represent geometric stimulation regions. They do not contain fixation, noise, object appearance, disparity values, or optical delays. The log set is not a complete BIDS dataset.

--prt exports a BrainVoyager protocol from the same measured intervals. export-olts --output DIR copies the six original color tables; preview --all-positions --output DIR saves every unique aperture and a manifest.

13 Troubleshooting and operational checks

Symptom	Checks and response
Long flip intervals	Use full-screen GLFW at native resolution. Stop competing GPU work and test normal run length. Do not assume maximum priority helps.
Object epoch / prefetch error	Check the prepared movie, SSD throughput, and free space. Review count, size, and update period; repeat with a new run.
Image bank not found	Check paths relative to the defining JSON and the presence of images.npy. Convert MAT input once with convert-images.
Framebuffer / refresh error	Match size_px, screen ID, full-screen resolution, OS scaling, and refresh to the apparatus.
Acquisition directory exists	Default: numbered backup. The error policy cancels; an active run must finish first.
Eyes / disparity reversed	Occlude each eye and verify mode, mirroring, offsets, and hardware eye assignment.

Checks before acquisition

Check stimulus settings and positions, cumulative timing, eye assignment, tasks, result saving and backups, and PRT/OLT output under the conditions used for your experiment. Run the automated software tests with python -m unittest discover -s tests -v.

Presentation requires hardware OpenGL. It may be unavailable through Remote Desktop; use a GPU-enabled local desktop. Verify timing and normal-run performance with the scanner, displays, and response devices used for data collection.

Further documentation and terms

README.md covers the project, background, and references. docs/USAGE.ja.md gives operating instructions; docs/PRESENTATION_TIMING.ja.md explains timing. docs/PTB3_FEATURES.ja.md and docs/PTB3_PARAMETER_MAP.ja.md describe feature and setting correspondence with the original. docs/PTB3_PARITY.ja.md explains starting positions, scan order, and compatibility.

Original BSD conditions are retained in LICENSE.txt. NOTICE.md contains source-image and dependency attribution. Conversion does not change database terms. Cite the original Retinotopy, the PyRetinotopy version, PsychoPy, and the applicable image datasets.

Original: github.com/hiroshiban/Retinotopy
PsychoPy: psychopy.org
GLFW plugin: github.com/psychopy/psychopy-glfw

14 Citation, references, and license

Citing the software

Report the PyRetinotopy version or commit and credit Hiroshi Ban, together with the original Retinotopy toolbox by Hiroshi Ban. Cite PsychoPy according to the version used. Original repository: https://github.com/hiroshiban/Retinotopy

When using object images

Conversion does not alter attribution or usage terms. Cite the applicable database papers. Relevant database publications are listed below.

1. Konkle, T., & Caramazza, A. (2013). Tripartite Organization of the Ventral Stream by Animacy and Object Size. Journal of Neuroscience, 33(25), 10235-10242.

2. Konkle, T., & Oliva, A. (2012). A real-world size organization of object responses in occipito-temporal cortex. Neuron, 74(6), 1114-1124.

3. Brady, T. F., Konkle, T., Alvarez, G. A., & Oliva, A. (2008). Visual long-term memory has a massive storage capacity for object details. PNAS, 105(38), 14325-14329.

4. Konkle, T., Brady, T. F., Alvarez, G. A., & Oliva, A. (2010). Conceptual distinctiveness supports detailed visual long-term memory. Journal of Experimental Psychology: General, 139(3), 558-578.

5. Konkle, T., & Oliva, A. (2012). A Familiar Size Stroop Effect: Real-world size is an automatic property of object representation. Journal of Experimental Psychology: Human Perception and Performance, 38, 561-569.

Research background of the original package

Yamamoto, H., Ban, H., et al. (2009). Large- and Small-Scale Functional Organization of Visual Field Representation in the Human Visual Cortex. In Visual Cortex: New Research, 195-226.

Ban, H., Yamamoto, H., et al. (2013). Topographic processing of an occluded object and the effects of spatiotemporal contexts in human early visual areas. Journal of Neuroscience, 33(43), 16992-17002. DOI: 10.1523/JNEUROSCI.1455-12.2013

Chou, I. W. Y., Ban, H., & Chang, D. H. F. (2021). Modulations of depth responses in the human brain by object context: Does biological relevance matter? eNeuro. DOI: 10.1523/ENEURO.0039-21.2021

Distribution terms

The original BSD conditions, copyright notice, and disclaimer are retained. See LICENSE.txt and NOTICE.md. Image data, PsychoPy, GLFW, and other dependencies retain their own terms.

15 Acknowledgments

Development of PyRetinotopy and its predecessor, Retinotopy

PyRetinotopy and its predecessor, Retinotopy, build on a series of modified versions of the retinotopic stimuli that I developed and used while working at the University of Birmingham, UK, under the supervision of Dr Andrew Welchman and Dr Zoe Kourtzi. I am deeply grateful to both of them for their support.

These stimuli were based on code originally developed by Dr Hiroki Yamamoto and Dr Yoshimichi Ejima at Kyoto University. That code ran on Solaris and IRIX workstations and was written in C/C++ using the Visualization Toolkit (VTK; Kitware Inc.). I learned a great deal from them about how to create visual stimuli.

I thoroughly revised and updated those stimuli in MATLAB and Psychtoolbox to create the original Retinotopy package. Over the years, I also added new functionality and new types of stimuli.

As Python has become increasingly widely used in psychology and neuroscience, I rewrote the original Retinotopy stimuli, developed in MATLAB and Psychtoolbox 3, in Python so that they run in PsychoPy. These rewritten stimuli form the PyRetinotopy package.

Psychtoolbox and PsychoPy

We thank the Psychtoolbox developers and contributors who supported the original package: David Brainard, Denis Pelli, Allen Ingling, Mario Kleiner, Richard F. Murray, Diederick C. Niehorster, and Tobias Wolf. We also thank the PsychoPy developers and contributors for the tools underlying presentation and response recording in the Python adaptation.

The optional GLFW path uses the official psychopy-glfw plugin by Matthew Cutone / Open Science Tools. We also acknowledge the developers of NumPy, Pillow, and the other open-source tools used for conversion, testing, and documentation.

Image databases and m-sequences

We sincerely thank Talia Konkle's research group and collaborators for developing and distributing the object-image databases. Research using these images should cite the applicable papers on the preceding page.

I thank Giedrius T. Buracas for the original m-sequence code and Thomas Liu for extending it. I subsequently made further modifications to that code.

Generative AI support for the Python/PsychoPy adaptation

PyRetinotopy was developed by rewriting the MATLAB Psychtoolbox 3 Retinotopy package in Python for PsychoPy, with support from OpenAI generative AI tools, including Codex. This support was used in code review, design, implementation, testing, and documentation.

We hope this software helps research and learning in visual science and neuroscience.

Hiroshi Ban, author and responsible investigator of PyRetinotopy, reviews the transfer of functionality from the original MATLAB/PTB3 Retinotopy and performs operational checks throughout development. He is responsible for the software and its release.